Version Control System

Nazih Errahel

erraheln@ex.istu.edu

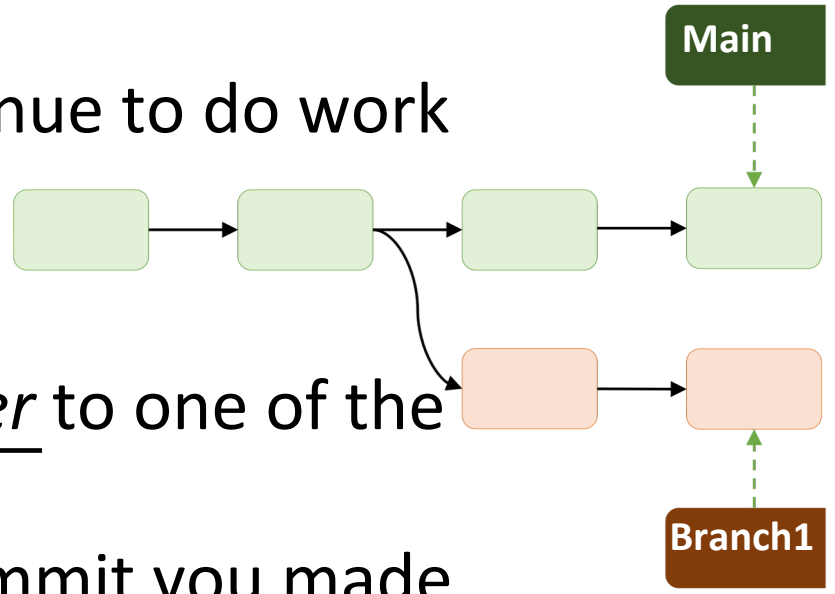
A quick recap…

Branching

Branching

Branch

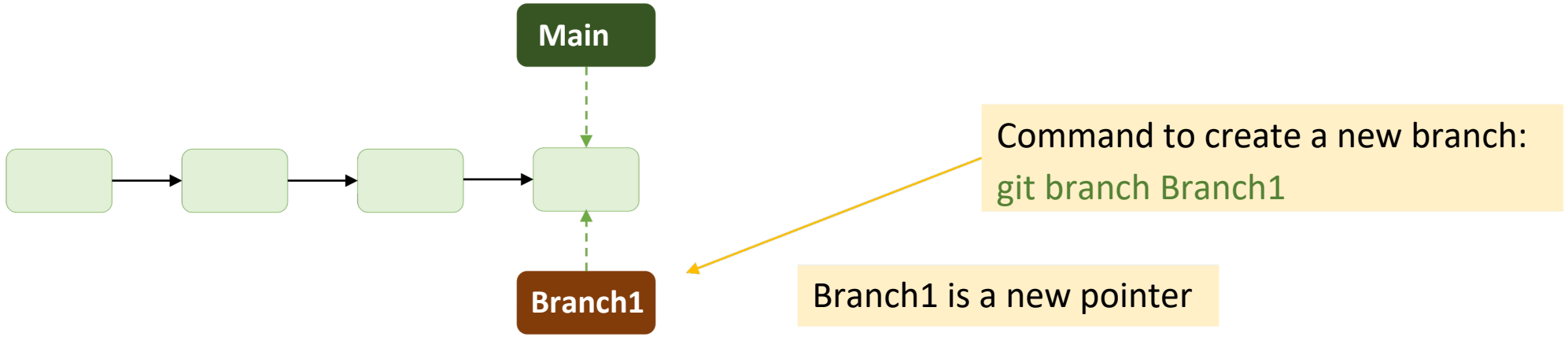
- A branch is a version of a project's working tree.
- diverge from the main line of development and continue to do work without messing with that main line.
- The default branch name in Git is master(or main)
- A branch in Git is simply a lightweight movable *pointer* to one of the commits.
- The main/master branch always points to the last commit you made.
- upon each new **commit**, the master branch pointer moves forward automatically.
- master branch in Git is not a special branch. It is exactly like any other branch.



Branching –create a new one

Branch

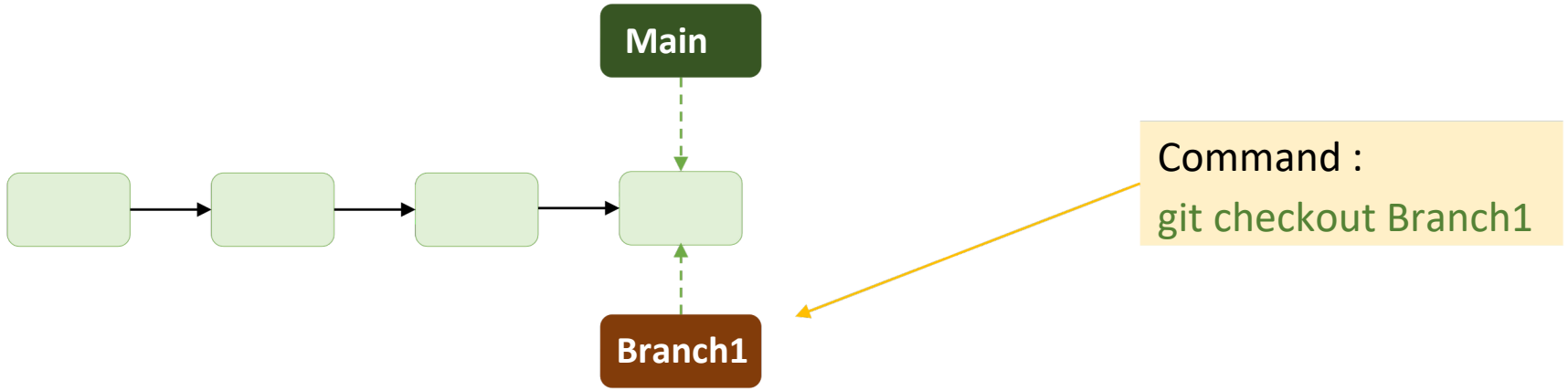
- A new branch means -> a new pointer



Branching –working on new branch

Branch

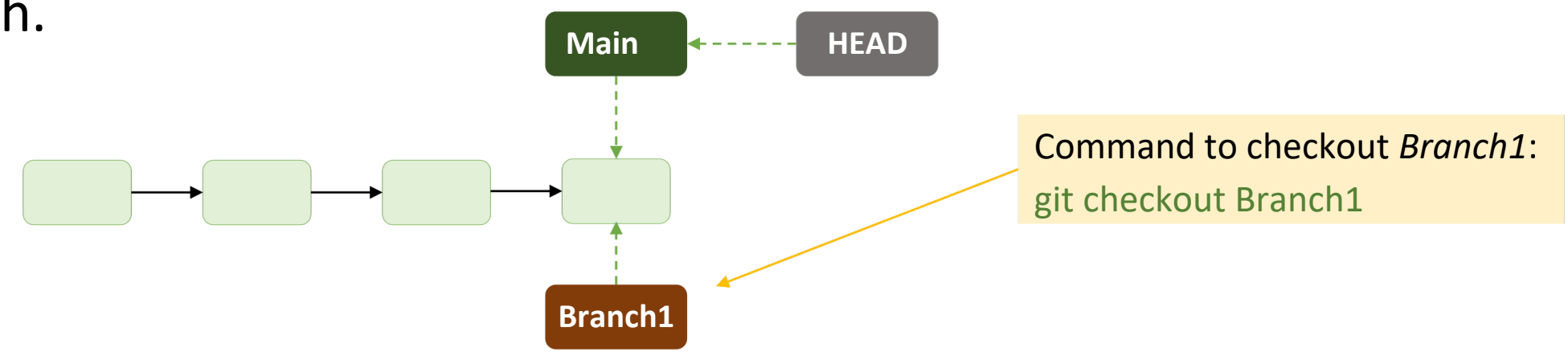
- Now how to switch to that new branch i.e. *Branch1?*
- *git checkout* command can be used to do so.
  - *git checkout Branch1*



Branching –working on new branch

Branch

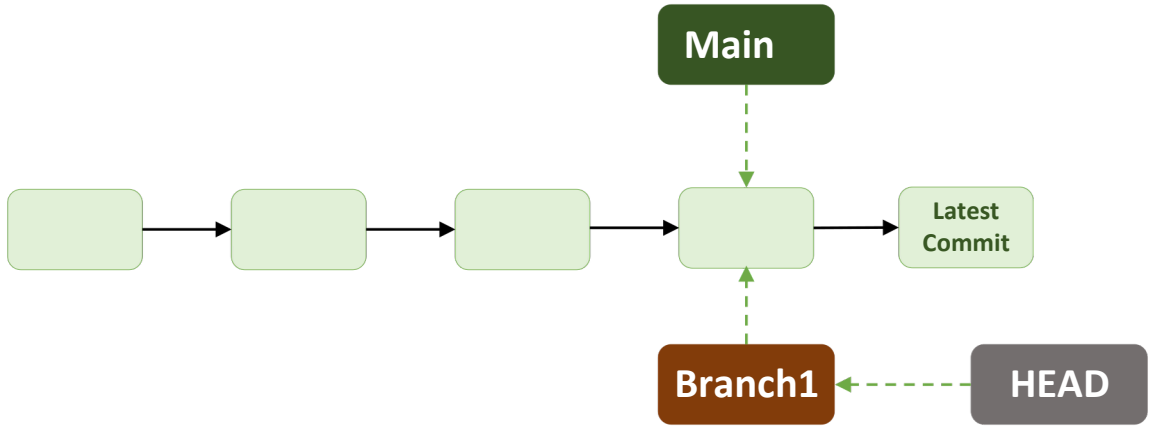
- *HEAD* is a *special pointer* to know what branch you are currently working on?
- When you start a new repo, *HEAD* pointer always points to *master* branch
- Upon checkout to different branch, HEAD pointer points to the new branch.



Branching –working on new branch

Branch

What happen when we make some changes to *Branch1?*



Make some modification

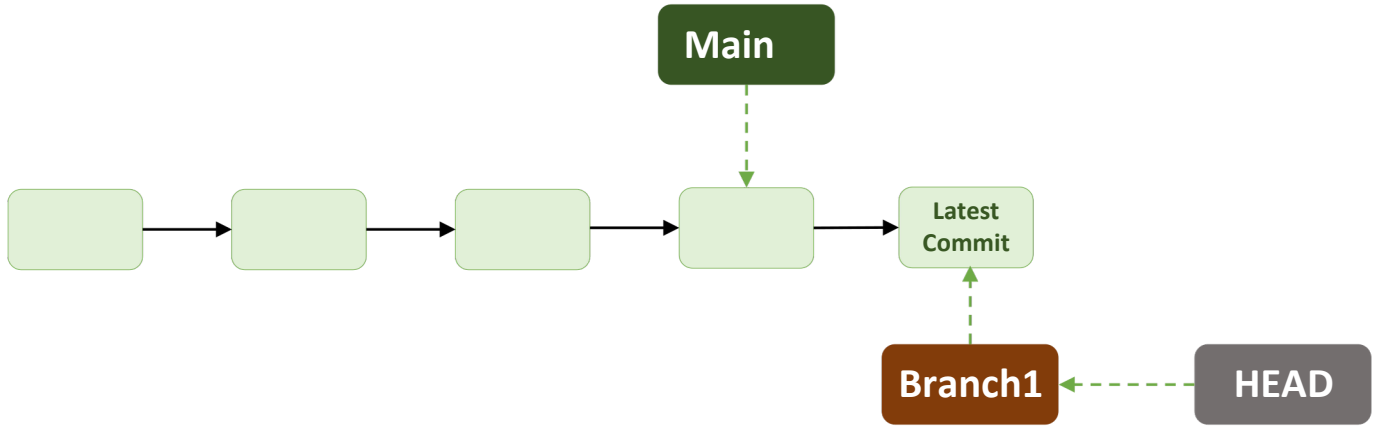
Branching –working on new branch

Branch

What happen when we make some changes to *Branch1?*

DIY: Try running (with multiple branches and multiple commits)

git log --all --graph



Pulling vs Cloning

Pulling vs cloning Repository

- git clone:
  - To get a local copy of an existing repository to work on
  - Clones a repository into a newly created directory, creates remote-tracking branches for each branch in the cloned repository
  - usually only used once for a given repository
  - usually to get a clean copy

- git pull command:
  - Incorporates changes from a remote repository into the current branch
  - ***update*** the local copy with new commits from the remote repository.

Fork and Merge

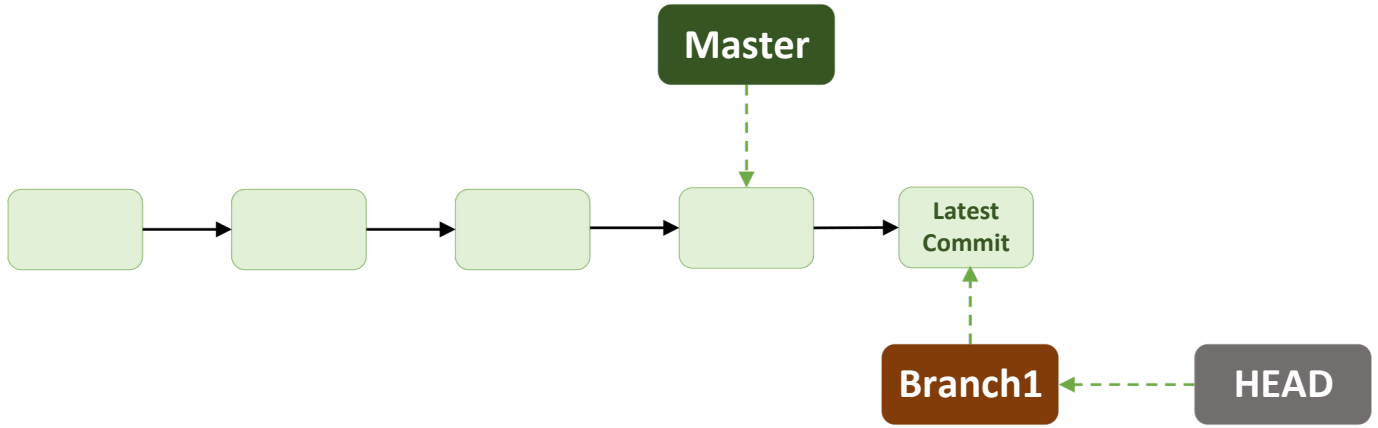
Forking

- Branching and forking provide two ways of diverging from the main code line.
- *Fork* is another way of saying clone or copy.
- Fork is a clone on the GitLab (or GitHub) side (it clones everything).
- If you fork a repository, you get that repository and all of its branches.
- A fork is independent from the original repository
- If the original repository is deleted, the fork remains.

Merging

In this case, lets merge *Branch1* and *Master* branch.

git checkout master



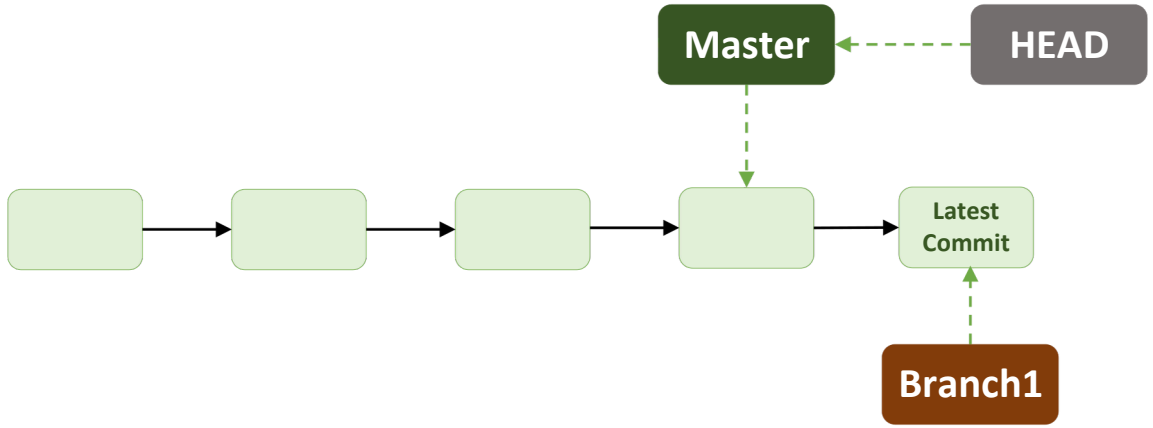
Merging

In this case, lets merge *Branch1* and *Master* branch.

git checkout master

git merge Branch1

Also known as fast-forward merging

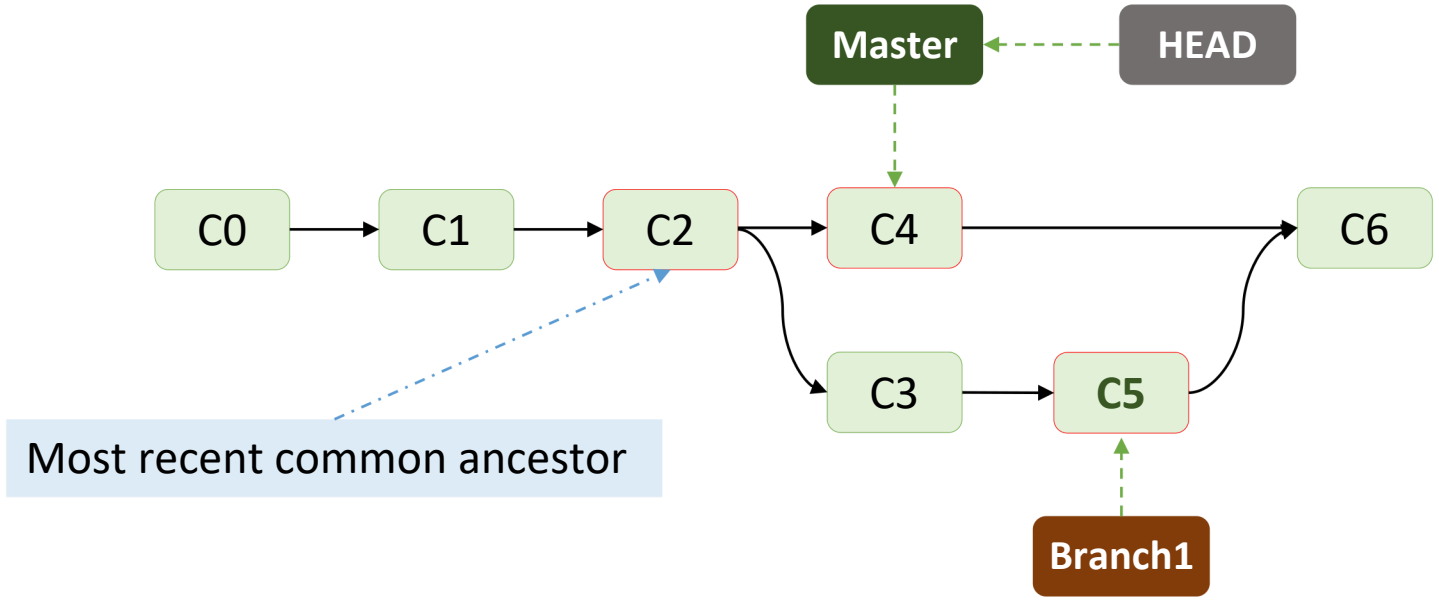


Merging

In this case, lets merge *Branch1* and *Master* branch.

git checkout master

git merge Branch1



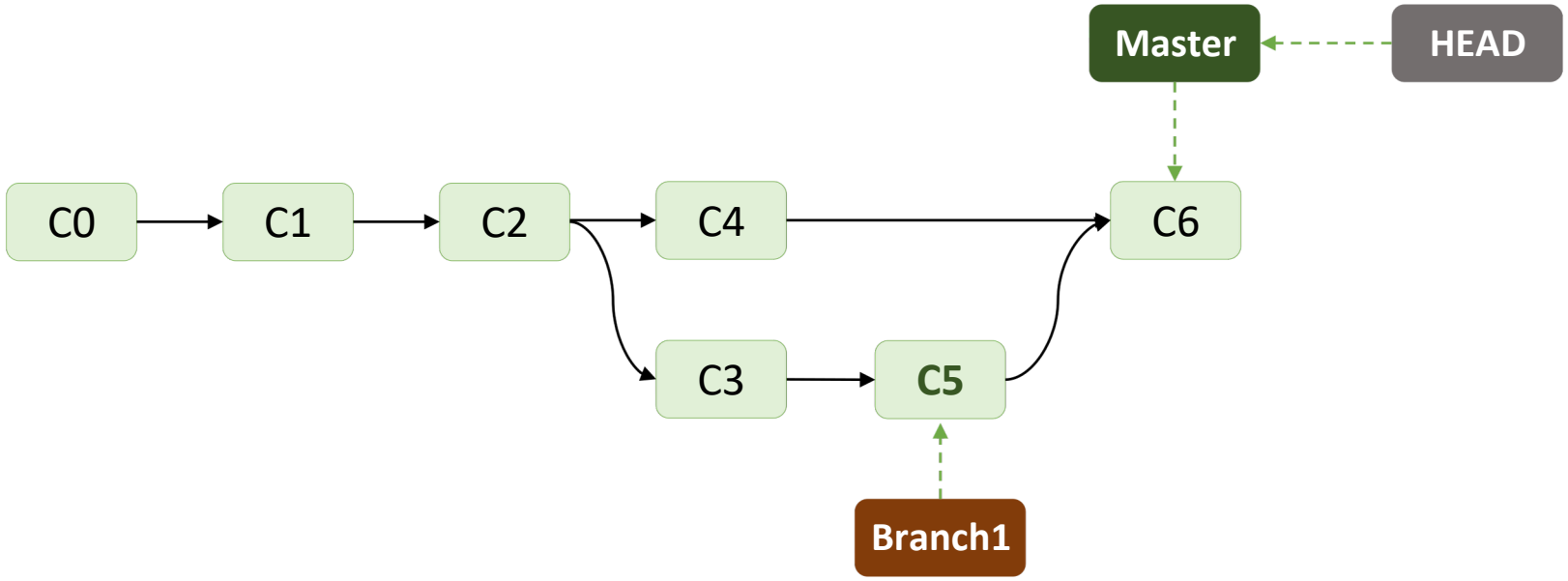
Merging

In this case, lets merge *Branch1* and *Master* branch.

git checkout master

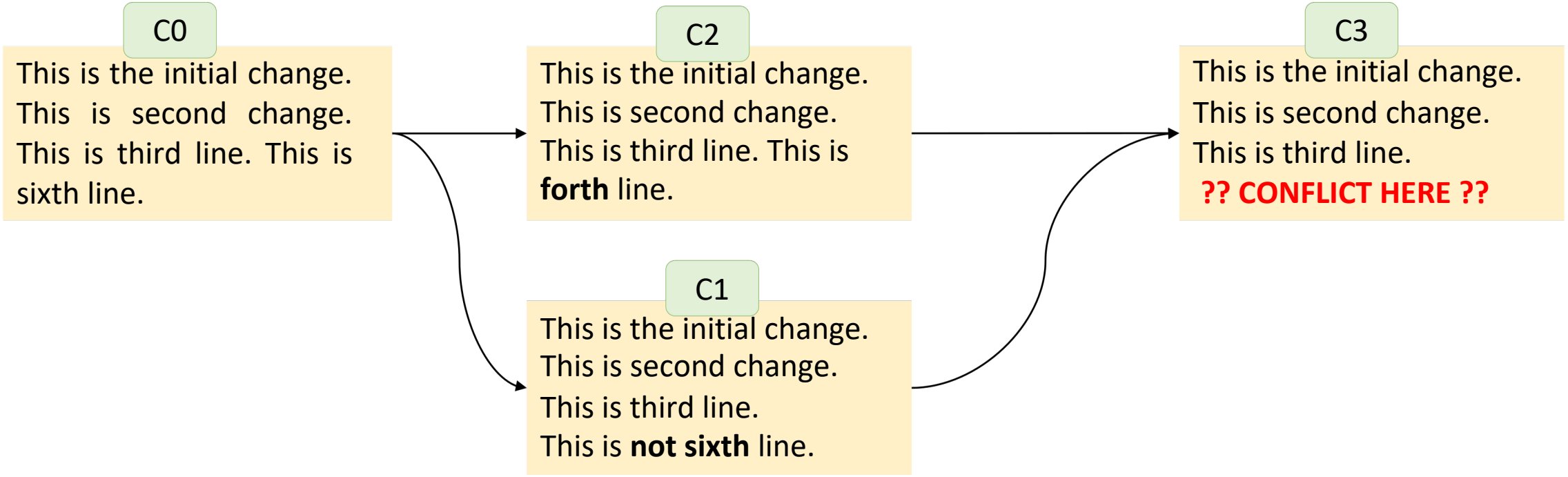
git merge Branch1

Also known as three-way merge



Merge conflict Example

- If you have changed the *same part* oft he same file differently in the two branches



Pull request vs Merge Request

Pull request vs Merge Request

According to GitLab Doc-

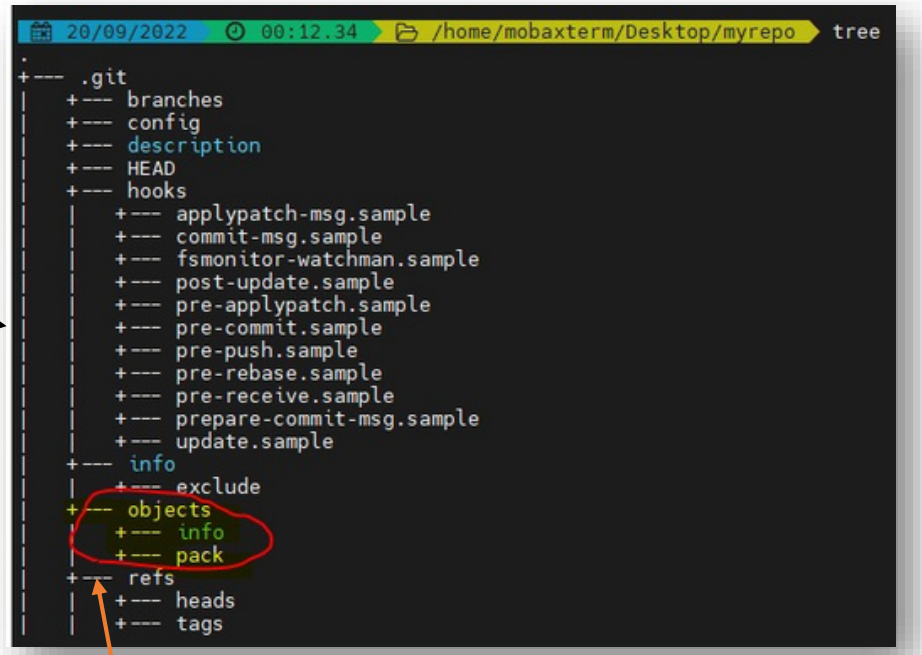
> *"…Tools such as GitHub and Bitbucket choose the name "pull request", because the first manual action is to pull the feature branch. Tools such as GitLab and others choose the name "merge request", because the final action is to merge the feature branch…." [src]*

https://docs.gitlab.com/ee/topics/gitlab_flow.html

Git objects

How git stores objects?

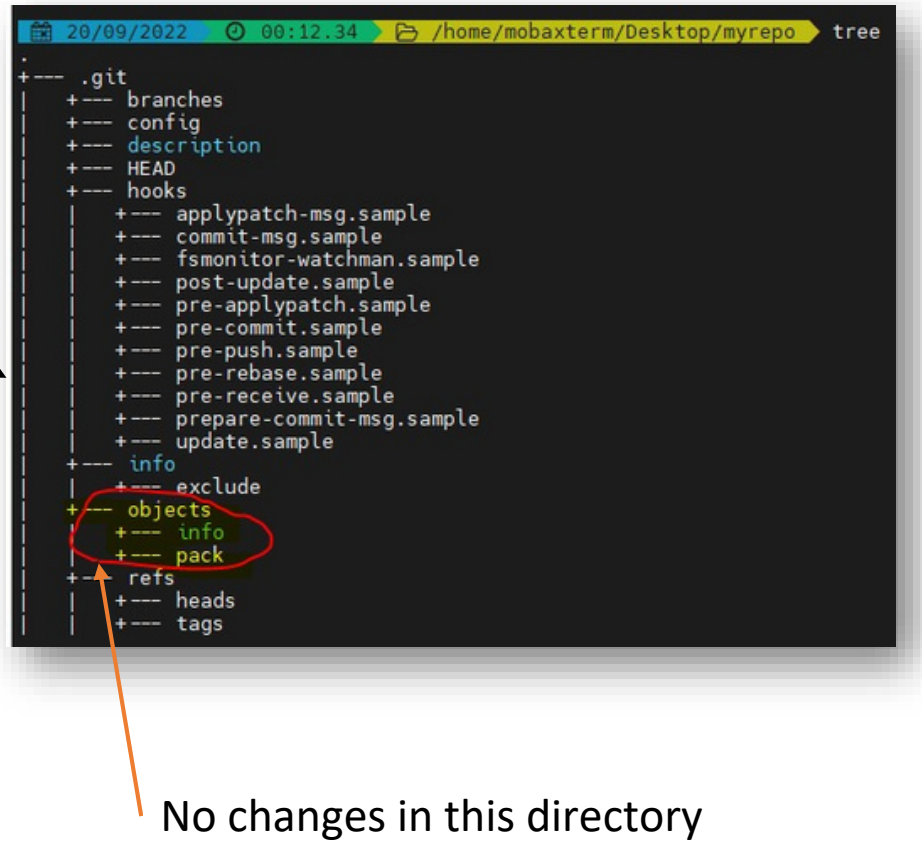
- `mkdir myrepo`
- `cd myrepo`
- `git init`



Let's focus on **.git/objects** directory

How git stores objects?

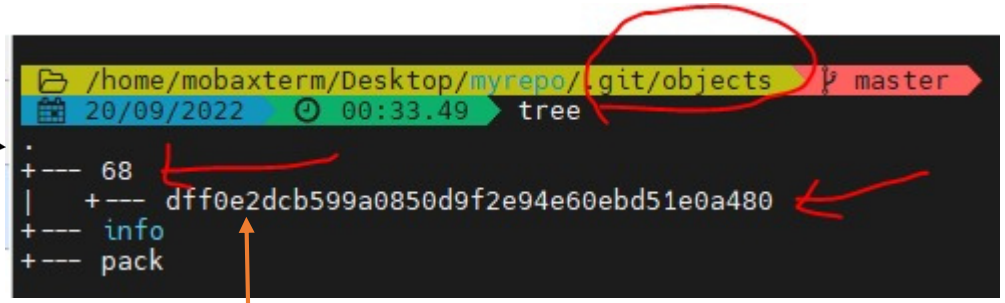
- mkdir
- myrepo cd
- myrepo
- git init
- echo "File1: This is line 1." > File1.txt



No changes in this directory

How git stores objects?

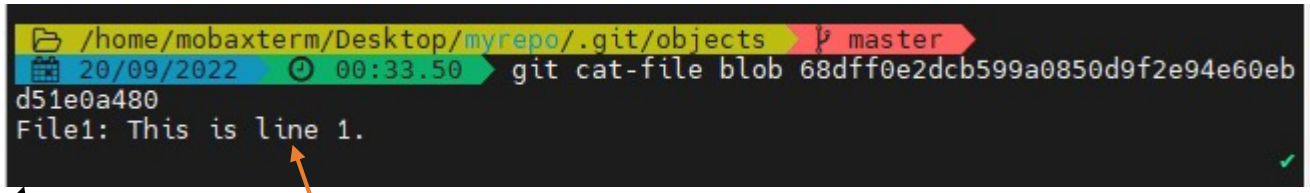
- `mkdir myrepo`
- `cd myrepo`
- `git init`
- `echo "File1: This is line 1." > File1.txt`
- `git add *`



New directory **68** and a file with alpha numeric file name

How git stores objects?

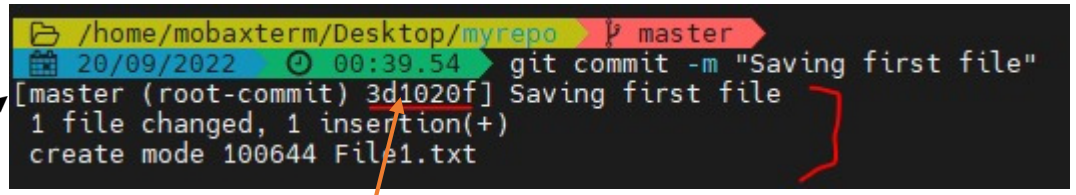
- mkdir myrepo
- cd myrepo
- git init
- echo "File1: This is line 1." > File1.txt
- git add *
- git cat-file blob 68dff0e2dcb599a0850d9f2e94e60ebd51e0a480



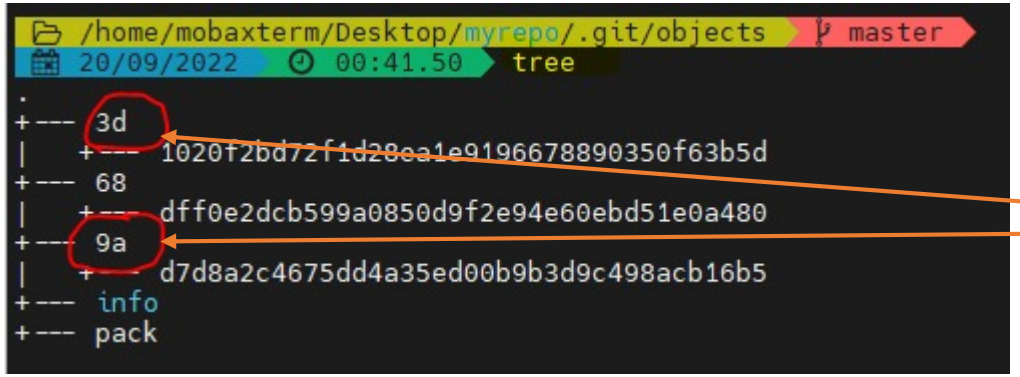
Showing the content of blob (or the file)

How git stores objects?

- `mkdir myrepo`
- `cd myrepo`
- `git init`
- `echo "File1: This is line 1." > File1.txt`
- `gitadd *`
- `git cat-file blob 68dff0e2dcb599a0850d9f2e94e60ebd51e0a480`
- `git commit -m "Saving first file"`



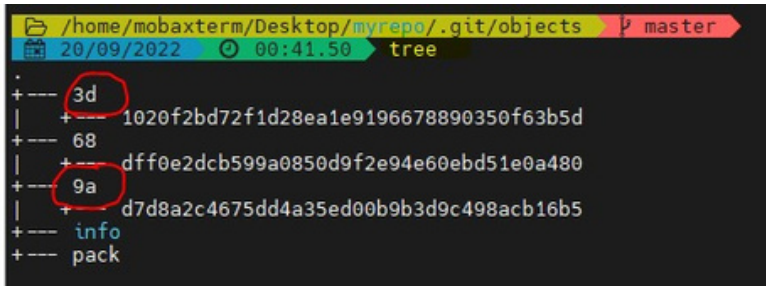
Notice the commit ID.

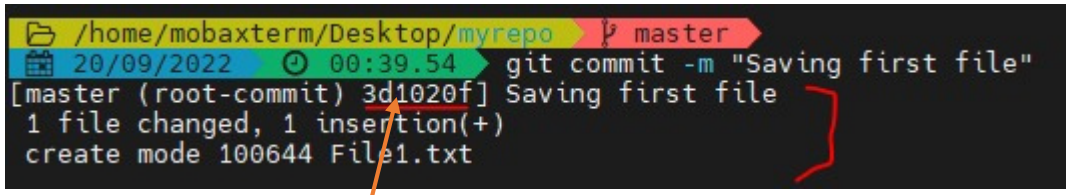


Notice the new directories and blobs

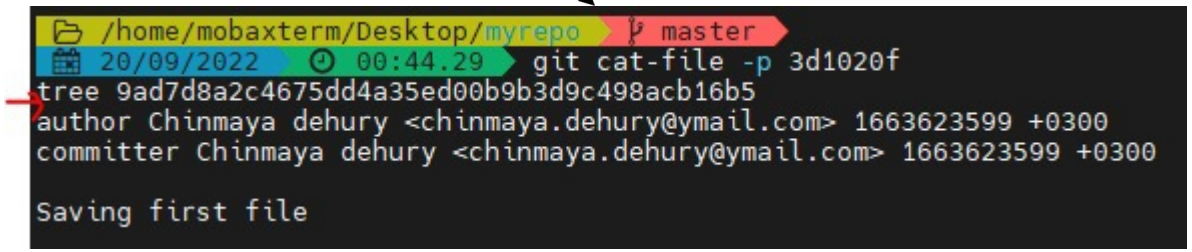
How git stores objects?

- mkdir myrepo
- cd myrepo
- git init
- echo "File1: This is line 1." > File1.txt
- gitadd *
- git cat-file blob 68dff0e2dcb599a0850d9f2e94e60ebd51e0a480
- git commit -m "Saving first file"
- git cat-file -p 3d1020f



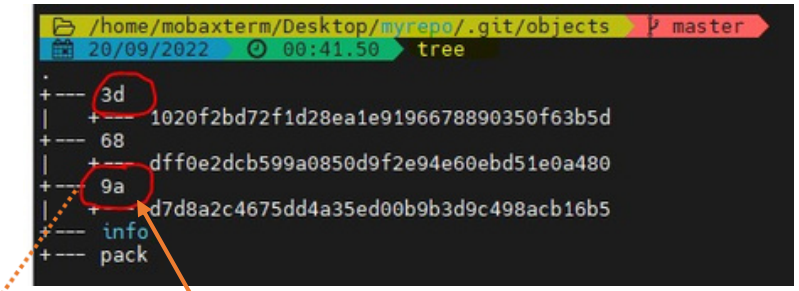


Notice the commit ID.



How git stores objects?

- mkdir myrepo
- cd myrepo
- git init
- echo "File1: This is line 1." > File1.txt
- git add *
- git cat-file blob 68dff0e2dcb599a0850d9f2e94e60ebd51e0a480
- git commit -m "Saving first file"
- git cat-file -p 3d1020f
- git cat-file -p 9ad7d8a2c46

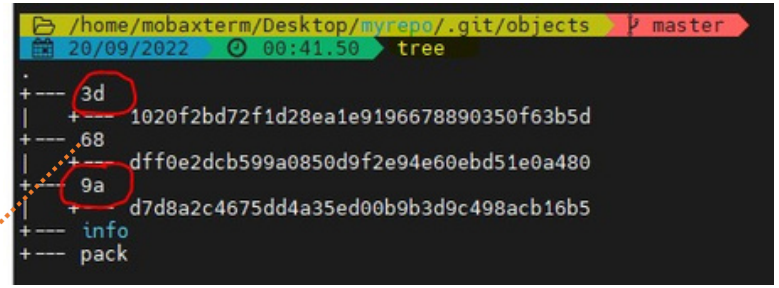


Notice the tree (NOT the command in the screenshot) ID.

Notice the blob ID.

How git stores objects?

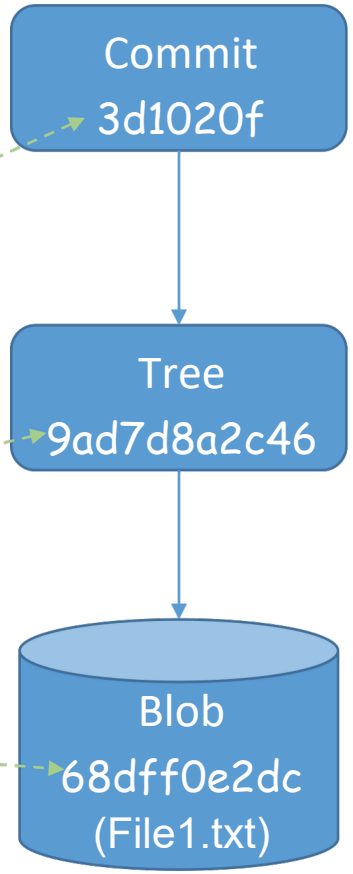
- mkdir myrepo
- cd myrepo
- git init
- echo "File1: This is line 1." > File1.txt
- git add * gitcat-file blob 68dff0e2dcb599a0850d9f2e94e60ebd51e0a480
- git commit -m "Saving first file"
- git cat-file -p 3d1020f
- git cat-file -p 9ad7d8a2c46
- git cat-file -p 68dff0e2dc



Notice the blob content.

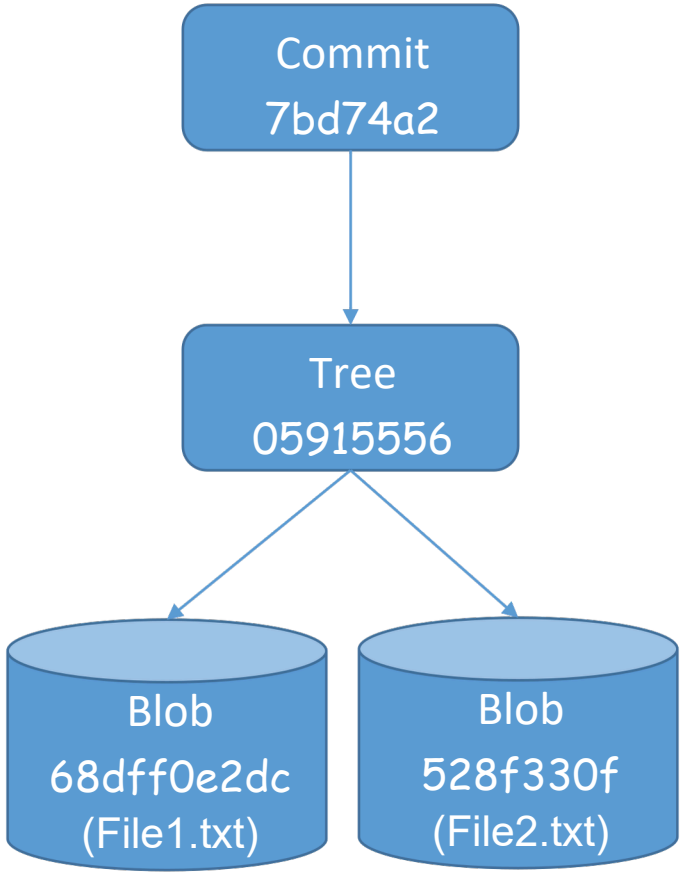
How git stores objects?

- `mkdir myrepo`
- `cd myrepo`
- `git init`
- `echo "File1: This is line 1." > File1.txt`
- `git add *`
- `git cat-file blob 68dff0e2dcb599a0850d9f2e94e60ebd51e0a480`
- `git commit -m "Saving first file"`
- `git cat-file -p 3d1020f`
- `git cat-file -p 9ad7d8a2c46`
- `git cat-file -p 68dff0e2dc`



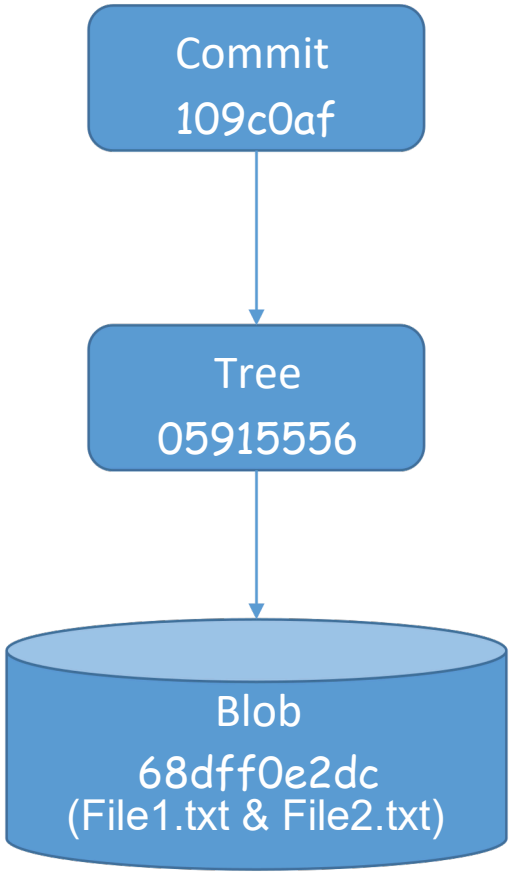
How git stores objects?

- `mkdir myrepo2`
- `cd myrepo2`
- `git init`
- `echo "File1: This is line 1." > File1.txt`
- `echo "File2: This is line 1." > File2.txt`
- `git add *`
- `git commit -m "Saving two files for the first time"`



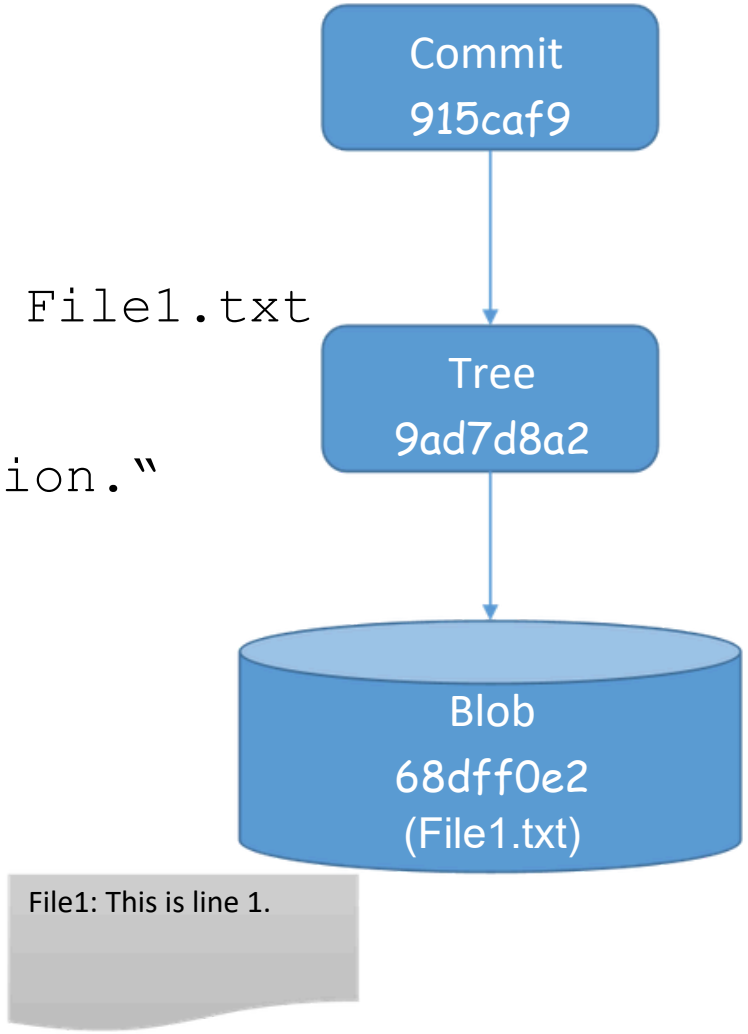
How git stores objects? **(Files with same content)**

- `mkdir myrepo2`
- `cd myrepo2`
- `git init`
- `echo "File1: This is line 1." > File1.txt`
- `echo "File1: This is line 1." > File2.txt`
- `git add *`
- `Git commit -m "files with same content."`



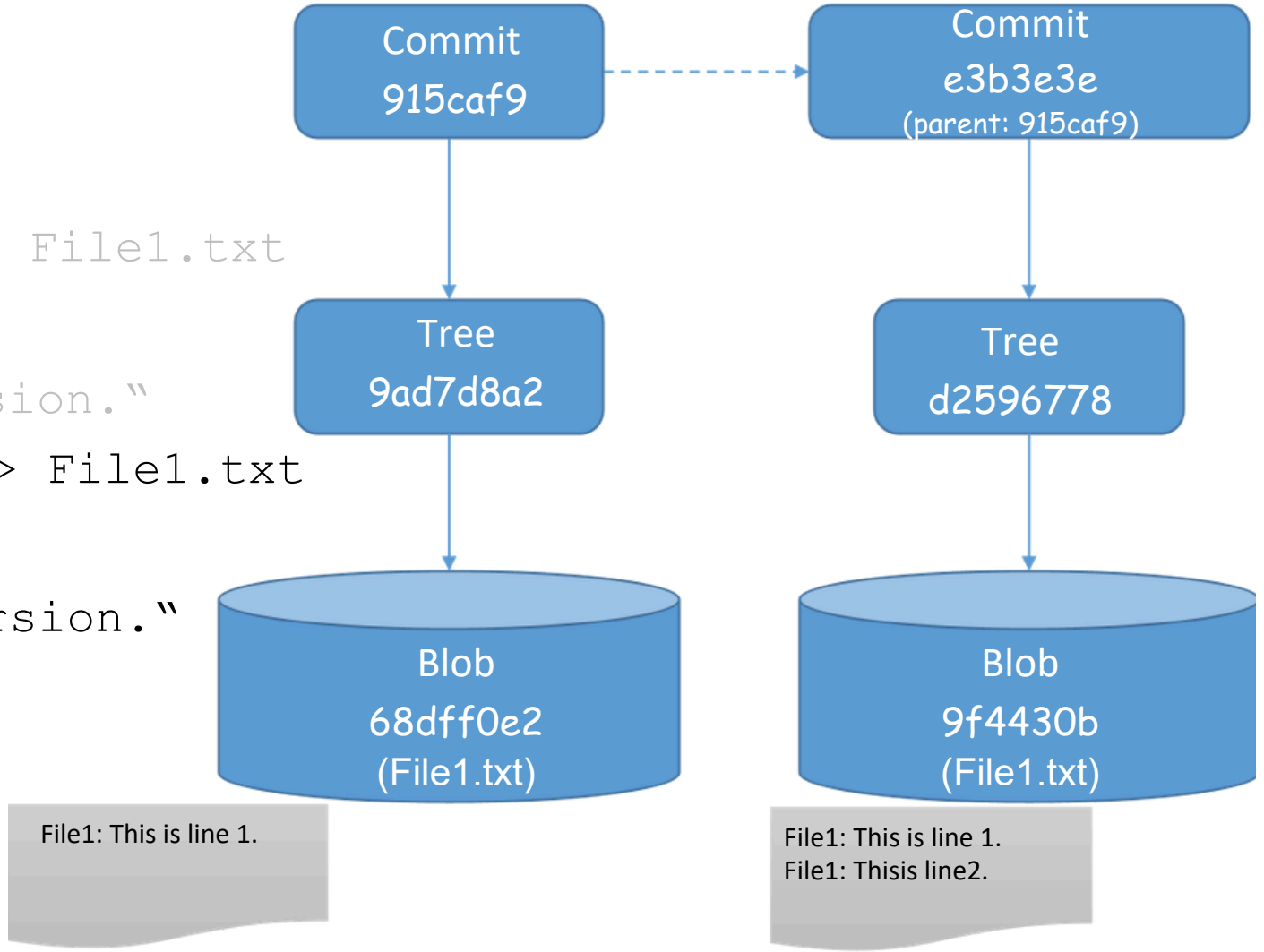
How git stores objects: Modified files?

- `mkdir myrepo4`
- `cd myrepo4`
- `git init`
- `echo "File1: This is line 1." > File1.txt`
- `git add *`
- `Git commit –m “Saving first version.“`



How git stores objects : Modified files?

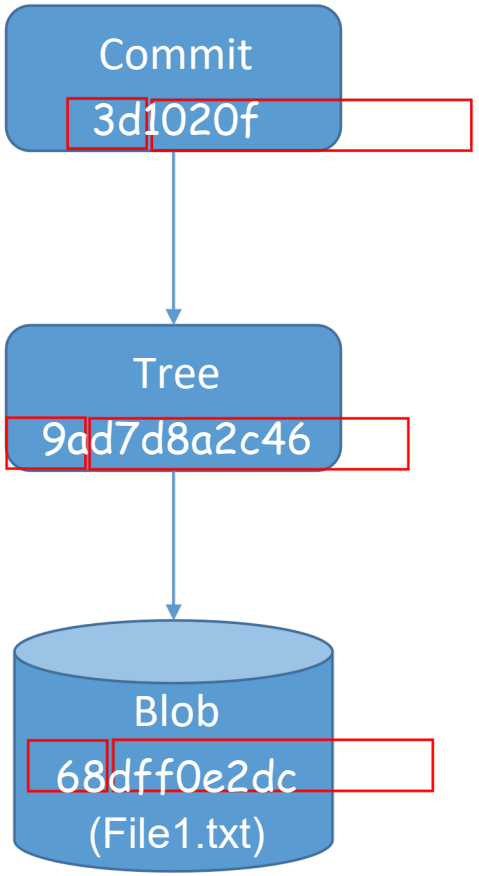
- mkdir myrepo4
- cd myrepo4
- git init
- echo "**File1:** This is line 1." > File1.txt
- git add *
- Git commit –m "Saving first version."
- echo "**File1:** This is line 2." >> File1.txt
- git add *
- Git commit –m "Saving Second version."



Git objects : Two-level directory structure

- mkdir myrepo
- cd myrepo
- git init
- echo "File1: This i
- git add *
- git cat-file blob 68dff0e2dcb599a085
- git commit -m "Saving first file"
- gitcat-file -p3d 1020f
- gitcat-file -p 9a d7d8a2c46
- gitcat-file -p 68 dff0e2dc

```
|   |   |   |   +--- master
|   +--- objects
|   |   +--- 3d
|   |   |   +--- 1020f2bd72f1d28ea1e9196678890350f63b5d
|   |   +--- 68
|   |   |   +--- dff0e2dcb599a0850d9f2e94e60ebd51e0a480
|   |   +--- 9a
|   |   |   +--- d7d8a2c4675dd4a35ed00b9b3d9c498acb16b5
|   |   +--- info
|   |   +--- pack
```



Directory name

File Name

Git object: handling branch & merge

DIY: Where is the branch info?

- Look at **.git/refs/heads/** directory

DIY: What happen when you merge two branches without conflict?

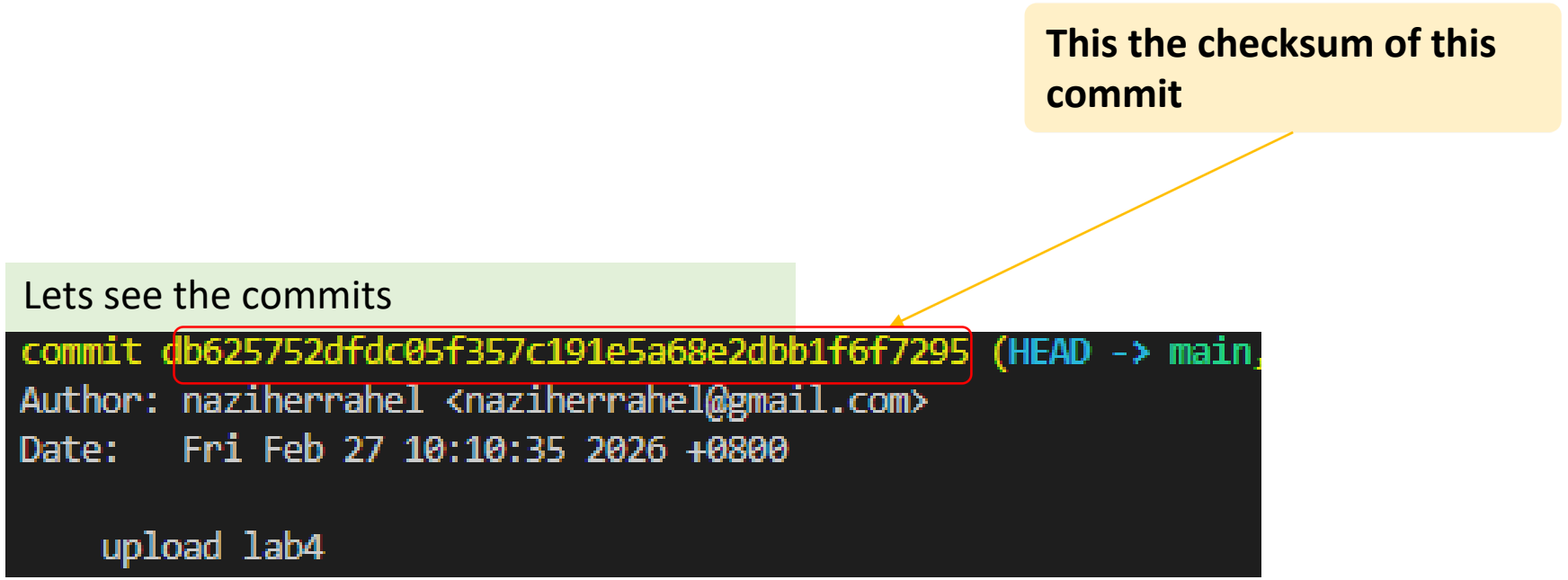
- Observe **.git/refs/heads/** directory

DIY: What happen when you merge two branches **with** conflict?

- Observe **.git/refs/heads/** directory

Git objects : Data Integrity

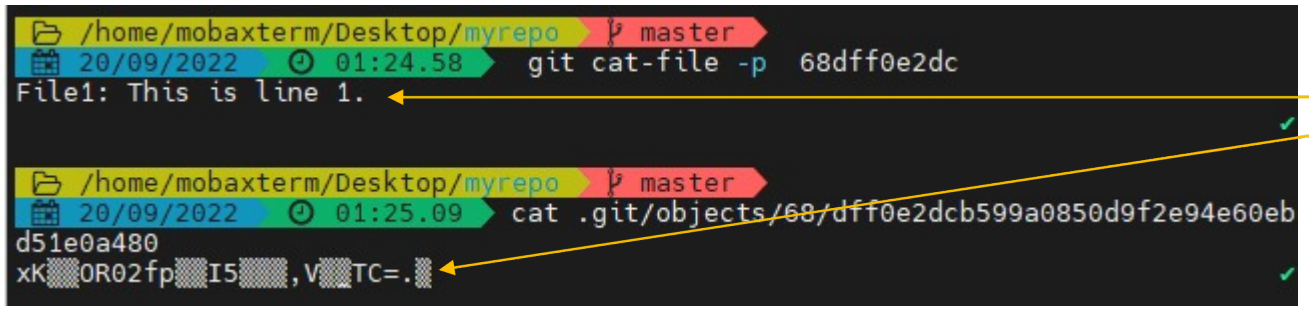
- Everything in Git is *checksummed* before it is stored and is then referred to by that checksum.



Git objects : Data Integrity

- Everything in Git is *checksummed* before it is stored and is then referred to by that checksum.
- Git stores everything in its database *not by file name* but by the hash value of its contents.
- a 40-character string
- composed of hexadecimal characters (0–9 and a–f)
- calculated based on the contents of a file or directory structure in Git

Git objects : Compression



Compare two ways to display the content

- Git compresses the content of a blob object (file)
  - Uses *zlib* tool
- The *commit* and *tree* content are very specifically formatted
  - Header of these files usually begins with *commit*or *tree*

Src: https://git-scm.com/book/en/v2/Git-Internals-Git-Objects

Summary

- Version Control System
- VCS Types & Goals
- VCS software tools
- Git --Installation & Configuration
- Git Repository and States
- Git data Integrity
- Some more git concepts….(Pulling vs cloning Repository, Branching, Forking, Merging, Rebase)

Further Reading…

Git Large File System (Git LFS)

- How to handle large files, for example the assets of Game development

Git Virtual File System (GVFS)

- Would you prefer to work with a Git repo having more than 3.5M files
- https://devblogs.microsoft.com/bharry/the-largest-git-repo-on-the-planet/

Git for large project:

- Very old project
- Large file count, activity, file size